الفحص والتمحيص في كود الحريص

Lecture 2 — Application Security Testing (AST): SAST and DAST

> **Application Security Testing (AST)** : making applications resilient against cyber threats by **identifying, analyzing and fixing** security vulnerabilities.

> **SAST (Static Application Security Testing)** : an **inside-out, white-box** approach that analyzes **source code at rest without executing it**.

> **DAST (Dynamic Application Security Testing)** : an **outside-in, black-box** approach that tests a **running application** by simulating real-world attacks against its endpoints (HTTP/APIs).

	SAST	**DAST**
View	Inside-out, **white-box**	Outside-in, **black-box**
Needs	Source code, build scripts, configs	A **live, running** target
Executes the app?	**No**	**Yes**
When in SDLC	**Early** (before compile/deploy)	Later (deployed / test environment)
Finds	Structural flaws: weak crypto, hard-coded secrets, unsafe patterns	Runtime behaviour: confirmed exploitable responses
Tool in the lecture	**Bandit** (Python)	Python `requests` script attacking a FastAPI app
Proof	"Possible" issue at a code line	Attack actually succeeds (or is defended)

Both are used together in **DevSecOps** pipelines.

SAST with Bandit

> **Bandit** : an open-source SAST tool that parses Python code into an **Abstract Syntax Tree (AST)** to find security bugs.

Q/ Which three vulnerabilities were planted in vulnerable_app.py ?

1. **Hard-coded secret** — CWE-259 (`DB_PASSWORD = "SuperSecretPassword123!"`).
2. **SQL injection vector** — CWE-89 (query built with an f-string).
3. **Command injection vector** — CWE-78 (`os.system(f"ping -c 1 {user_ip}")`).

Bandit ID	Issue	Severity	Confidence
B105 hardcoded_password_string	Possible hard-coded password	Medium	High
B608 sql_injection_string_vulnerability	SQL injection through string formatting	Medium	High
B605 start_process_with_a_shell	Process started with a shell, possible injection	**High**	High

Result: 3 issues (Medium: 2, High: 1) in 18 lines.

Q/ How was each flaw remediated (secure_app.py) ?

1. Secret → read from an **environment variable**: `os.getenv("DATABASE_PASSWORD")` .
2. SQL → **parameterized query** with `?` placeholders: `cursor.execute(query, (username, password))` .
3. Command → `subprocess.run(["ping", "-c", "1", user_ip])` — an **argument array, no shell**, so input is only an argument.

Re-scan: **No issues identified** (Medium 0, High 0).

DAST against a live API

Q/ What did the vulnerable FastAPI service expose ?

1. `/login` — builds the SQL query with an f-string.
2. `/ping` — runs `os.popen(f"ping -c 1 {ip}")` .

Colab note: the API runs on 127.0.0.1 **inside Google's machine**, so **Localtunnel** creates a public URL; the scanner sends the `bypass-tunnel-reminder` header to skip the splash page.

DAST probe	Payload	Vulnerable app	Secure app
SQL injection	username = `admin' OR '1'='1`, any password	**status: success** — "Welcome ('admin', 'SuperSecret123')"	**status: fail** — Invalid credentials
Command injection	ip = `127.0.0.1; cat /etc/hostname`	Shell runs the extra command	`ping: 127.0.0.1; cat /etc/hostname: Name or service not known` — treated as one argument

Q/ Why does `admin' OR '1'='1` log in ?

- The quote closes the string and `OR '1'='1'` makes the WHERE condition **always true**, so the first user (admin) is returned without the right password.

Q/ Why does `; cat /etc/hostname` work against os.popen ?

- The **shell** treats `;` as a command separator and runs a second command. With an argument list and no shell, `;` is just part of the hostname.

Q/ What is the DevSecOps lesson ?

- SAST catches the flaw **early in code**; DAST **confirms exploitability** on the running system; after fixing, both re-scans pass (SAST: no issues; DAST: attacks defended).

CWE	Name	Fix
CWE-259	Hard-coded password	Environment variables / secrets manager
CWE-89	SQL injection	Parameterized queries
CWE-78	OS command injection	Argument arrays, no shell (subprocess list)

Summary of Advanced Cybersecurity L2: Application Security Testing (SAST / DAST). Built only from the lecture file; check the original slides for anything you're unsure about.